

TLS Fingerprinting App Fixes

<https://gemini.google.com/app/e25700ec99bd44eb>

User prompt: Hello, I am working on this TLS fingerprinting project. When I hit stop live capture, the application just quits. I dont want that. Please tell me how to fix that, and if you need any additional files. Also can you reduce the opacity of the green and the blue? Too harsh.

Response: The application quits because PcapWorker and LiveCaptureWorker overwrite the built-in isRunning() method of QThread. By assigning a boolean value to self.isRunning = True, the GUI triggers an unhandled TypeError when window.py evaluates if self.liveWorker and self.liveWorker.isRunning(): during the stop sequence. No additional files are required to resolve this.

1. Rename the threading flag in workers.py Change the variable self.isRunning to self.isCapturing in both worker classes to prevent shadowing the PyQt method.

```
class PcapWorker(QThread):
    def __init__(self, pcapPath, dbPath):
        super().__init__()
        self.pcapPath = pcapPath
        self.dbPath = dbPath
        self.isCapturing = True

    def run(self):
        # ... existing setup code ...
        for result in readPcap(self.pcapPath, captureStats):
            if not self.isCapturing:
                break
        # ... existing finish code ...

    def stopCapture(self):
        self.isCapturing = False

class LiveCaptureWorker(QThread):
    def __init__(self, interfaceName, dbPath, bpfFilter, packetBuffer=None):
        # ... existing init code ...
        self.isCapturing = True

    def run(self):
        # ... existing setup code ...
        while self.isCapturing:
            self.msleep(100)
        # ... existing finish code ...
```

```
def stopCapture(self):
    self.isCapturing = False
```

How to verify: Start a live capture in the application and click the stop button; the process will now halt gracefully without crashing the window.

2. Reduce color opacity in window.py The QColor objects currently use an alpha value of 50, which dictates the transparency. Lowering this to 20 will significantly soften the visual impact. Apply this change to both the appendTableRow and labelSelectedFingerprint methods.

```
# In appendTableRow
isClient = packetData["role"] == "Client"
rowColor = QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20)

# In labelSelectedFingerprint
isClient = entry["role"] == "Client"
newItem.setBackground(QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20))
```

How to verify: Load a PCAP file to populate the table and visually confirm the green and blue rows are much more transparent.

User prompt: Just give me the two new complete codes

Response: **workers.py**

```
import os
from PyQt6.QtCore import QThread, pyqtSignal
from scapy.all import AsyncSniffer, IP, IPv6, TCP, get_working_ifaces
from src.capture import read_pcap as readPcap, TCPReassembler, CaptureStats
from src.ja3 import (
    compute_ja3_hash as computeJa3Hash,
    compute_ja3_string as computeJa3String,
    compute_ja3s_hash as computeJa3sHash,
    compute_ja3s_string as computeJa3sString
)
from src.ja4 import (
    compute_ja4_string as computeJa4String,
    compute_ja4s_string as computeJa4sString
)
from src.db import FingerprintDB

def clientPacketData(result, clientHello, fingerprintDb):
    ja3Hash = computeJa3Hash(clientHello)
    ja4Hash = computeJa4String(clientHello)
    ja3Match = fingerprintDb.lookup(ja3Hash, "ja3") or "Unknown"
    ja4Match = fingerprintDb.lookup(ja4Hash, "ja4") or "Unknown"
```

```

targetKind = "ja3" if ja3Match == "Unknown" else "ja4"
targetHash = ja3Hash if targetKind == "ja3" else ja4Hash
return {
    "role": "Client",
    "source": f"{result.src_ip}:{result.src_port}",
    "destination": f"{result.dst_ip}:{result.dst_port}",
    "sni": clientHello.server_name or "N/A",
    "ja3Hash": ja3Hash,
    "ja4Hash": ja4Hash,
    "ja3Match": ja3Match,
    "ja4Match": ja4Match,
    "fingerprintKind": targetKind,
    "fingerprintHash": targetHash,
    "ja3Raw": computeJa3String(clientHello),
}

```

```

def serverPacketData(result, serverHello, fingerprintDb):
    ja3sHash = computeJa3sHash(serverHello)
    ja4sHash = computeJa4sString(serverHello)
    ja3sMatch = fingerprintDb.lookup(ja3sHash, "ja3s") or "Unknown"
    ja4sMatch = fingerprintDb.lookup(ja4sHash, "ja4s") or "Unknown"
    targetKind = "ja3s" if ja3sMatch == "Unknown" else "ja4s"
    targetHash = ja3sHash if targetKind == "ja3s" else ja4sHash
    return {
        "role": "Server",
        "source": f"{result.src_ip}:{result.src_port}",
        "destination": f"{result.dst_ip}:{result.dst_port}",
        "sni": "N/A",
        "ja3Hash": ja3sHash,
        "ja4Hash": ja4sHash,
        "ja3Match": ja3sMatch,
        "ja4Match": ja4sMatch,
        "fingerprintKind": targetKind,
        "fingerprintHash": targetHash,
        "ja3Raw": computeJa3sString(serverHello),
    }

```

```

class PcapWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    captureFinished = pyqtSignal(str)
    errorOccurred = pyqtSignal(str)

    def __init__(self, pcapPath, dbPath):
        super().__init__()
        self.pcapPath = pcapPath
        self.dbPath = dbPath
        self.isCapturing = True

    def run(self):
        try:
            fingerprintDb = FingerprintDB(str(self.dbPath))
            captureStats = CaptureStats()

```

```

    if not os.path.exists(self.pcapPath) or os.path.getsize(self.pcapPath) < 24:
        self.errorOccurred.emit("Selected file is empty or corrupted.")
        return
    for result in readPcap(self.pcapPath, captureStats):
        if not self.isCapturing:
            break
        if result.client_hello:
            self.rowExtracted.emit(
                clientPacketData(result, result.client_hello, fingerprintDb)
            )
        if result.server_hello:
            self.rowExtracted.emit(
                serverPacketData(result, result.server_hello, fingerprintDb)
            )
        self.captureFinished.emit(captureStats.summary())
    except Exception as e:
        self.errorOccurred.emit(str(e))

def stopCapture(self):
    self.isCapturing = False

class LiveCaptureWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    errorOccurred = pyqtSignal(str)
    captureFinished = pyqtSignal(str)

    def __init__(self, interfaceName, dbPath, bpfFilter, packetBuffer=None):
        super().__init__()
        self.interfaceName = interfaceName
        self.dbPath = dbPath
        self.bpfFilter = bpfFilter
        self.captureStats = CaptureStats()
        self.reassembler = TCPReassembler(self.captureStats)
        self.packetBuffer = packetBuffer
        self.isCapturing = True
        self.sniffer = None

    def processLivePacket(self, pkt):
        if self.packetBuffer is not None:
            self.packetBuffer.append(pkt)
        if not (pkt.haslayer(TCP) and (pkt.haslayer(IP) or pkt.haslayer(IPv6))):
            return
        ipLayer = pkt[IP] if pkt.haslayer(IP) else pkt[IPv6]
        tcpLayer = pkt[TCP]
        srclp = str(ipLayer.src)
        dstlp = str(ipLayer.dst)
        srcPort = int(tcpLayer.sport)
        dstPort = int(tcpLayer.dport)
        seq = int(tcpLayer.seq)
        flags = int(tcpLayer.flags)
        payload = bytes(tcpLayer.payload)
        timestamp = float(pkt.time)

```

```

results = self.reassembler.process_packet(
    srclp, dstlp, srcPort, dstPort, seq, flags, payload, timestamp
)
for res in results:
    if res.client_hello:
        self.rowExtracted.emit(
            clientPacketData(res, res.client_hello, self.fingerprintDb)
        )
    if res.server_hello:
        self.rowExtracted.emit(
            serverPacketData(res, res.server_hello, self.fingerprintDb)
        )

def run(self):
    try:
        self.fingerprintDb = FingerprintDB(str(self.dbPath))
        if self.interfaceName == "Any":
            ifaceArg = [iface.network_name for iface in get_working_ifaces()]
        elif self.interfaceName == "Default":
            ifaceArg = None
        else:
            ifaceArg = self.interfaceName
        self.sniffer = AsyncSniffer(
            iface=ifaceArg,
            filter=self.bpfFilter,
            prn=self.processLivePacket,
            store=False
        )
        self.sniffer.start()
        while self.isCapturing:
            self.msleep(100)
        if self.sniffer and self.sniffer.running:
            self.sniffer.stop()
        self.captureFinished.emit(self.captureStats.summary())
    except Exception as exceptionObject:
        self.errorOccurred.emit(f"Capture failed: {exceptionObject}")

def stopCapture(self):
    self.isCapturing = False

```

window.py

```

from pathlib import Path
import shutil
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QColor
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
    QTableWidgetItem, QTableWidgetItem, QFileDialog, QHeaderView,
    QComboBox, QLabel, QTextEdit, QInputDialog, QAbstractItemView,
    QLineEdit

```

```

)
from scapy.all import get_working_ifaces, wrpcap
from src.db import FingerprintDB
from workers import PcapWorker, LiveCaptureWorker

class TlsMonitorGui(QMainWindow):
    def __init__(self, pythonEngineDir):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1150, 650)
        self.dbPath = pythonEngineDir / "fingerprints.json"
        self.pcapsDir = pythonEngineDir.parent / "pcaps"
        self.fingerprintDb = None
        self.pcapWorker = None
        self.liveWorker = None
        self.rowCache = []
        self.capturedPackets = []
        self.loadedPcapPath = None
        mainWidget = QWidget()
        self.setCentralWidget(mainWidget)
        mainLayout = QVBoxLayout(mainWidget)
        controlLayout = QHBoxLayout()
        self.loadButton = QPushButton("Load PCAP")
        self.loadButton.clicked.connect(self.selectPcapFile)
        controlLayout.addWidget(self.loadButton)
        self.cancelLoadButton = QPushButton("Cancel Load")
        self.cancelLoadButton.clicked.connect(self.cancelPcapLoad)
        self.cancelLoadButton.setEnabled(False)
        controlLayout.addWidget(self.cancelLoadButton)
        self.saveButton = QPushButton("Save PCAP")
        self.saveButton.clicked.connect(self.savePcapFile)
        controlLayout.addWidget(self.saveButton)
        self.clearButton = QPushButton("Clear Records")
        self.clearButton.clicked.connect(self.clearRecords)
        controlLayout.addWidget(self.clearButton)
        self.refreshInterfacesButton = QPushButton("Refresh Interfaces")
        self.refreshInterfacesButton.clicked.connect(self.populateInterfaces)
        controlLayout.addWidget(self.refreshInterfacesButton)
        controlLayout.addWidget(QLabel("Interface:"))
        self.interfaceDropdown = QComboBox()
        self.populateInterfaces()
        controlLayout.addWidget(self.interfaceDropdown)
        controlLayout.addWidget(QLabel("BPF Filter:"))
        self.bpfFilterInput = QLineEdit("tcp port 443")
        controlLayout.addWidget(self.bpfFilterInput)
        self.liveButton = QPushButton("Start Live Capture")
        self.liveButton.clicked.connect(self.toggleLiveCapture)
        controlLayout.addWidget(self.liveButton)
        self.labelButton = QPushButton("Label Fingerprint")
        self.labelButton.clicked.connect(self.labelSelectedFingerprint)
        self.labelButton.setEnabled(False)
        controlLayout.addWidget(self.labelButton)

```

```

controlLayout.addStretch()
mainLayout.addLayout(controlLayout)
self.dataTable = QTableWidgetItem(0, 8)
self.dataTable.setHorizontalHeaderLabels([
    "Role", "Source", "Destination", "SNI", "JA3 Hash", "JA4 Hash",
    "JA3 Match", "JA4 Match"
])

header = self.dataTable.horizontalHeader()
header.setSectionResizeMode(QHeaderView.ResizeMode.ResizeToContents)
header.setSectionResizeMode(3, QHeaderView.ResizeMode.Stretch)
self.dataTable.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows)
self.dataTable.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
self.dataTable.itemSelectionChanged.connect(self.displayRowDetails)
mainLayout.addWidget(self.dataTable)
self.detailsPane = QTextEdit()
self.detailsPane.setReadOnly(True)
self.detailsPane.setFixedHeight(120)
self.detailsPane.setPlaceholderText("Select any handshake above to view raw metadata and JA3
string.")
mainLayout.addWidget(self.detailsPane)

def populateInterfaces(self):
    self.interfaceDropdown.clear()
    self.interfaceDropdown.addItem("Any (All Interfaces)", "Any")
    try:
        for iface in get_working_ifaces():
            self.interfaceDropdown.addItem(iface.name, iface.network_name)
    except Exception:
        self.interfaceDropdown.addItem("Default", "Default")

def closeEvent(self, event):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.pcapWorker.wait()
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveWorker.wait()
    event.accept()

def getDb(self):
    if self.fingerprintDb is None:
        self.fingerprintDb = FingerprintDB(str(self.dbPath))
    return self.fingerprintDb

def selectPcapFile(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        return
    filePath, _ = QFileDialog.getOpenFileName(
        self, "Open PCAP", str(self.pcapsDir), "PCAP Files (*.pcap *.pcapng)"
    )
    if filePath:

```

```

self.clearRecords()
self.loadedPcapPath = filePath
self.loadButton.setEnabled(False)
self.liveButton.setEnabled(False)
self.cancelLoadButton.setEnabled(True)
self.pcapWorker = PcapWorker(filePath, self.dbPath)
self.pcapWorker.rowExtracted.connect(self.appendTableRow)
self.pcapWorker.errorOccurred.connect(lambda err: self.detailsPane.setText(f"Error: {err}"))
self.pcapWorker.captureFinished.connect(self.onCaptureFinished)
self.pcapWorker.start()

```

```

def cancelPcapLoad(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.detailsPane.append("\nPCAP loading cancelled.")

```

```

def onCaptureFinished(self, statsSummary):
    self.loadButton.setEnabled(True)
    self.liveButton.setEnabled(True)
    self.cancelLoadButton.setEnabled(False)
    currentText = self.detailsPane.toPlainText()
    self.detailsPane.setText(f"{currentText}\n\nCapture Finished.\n{statsSummary}")

```

```

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        if self.pcapWorker and self.pcapWorker.isRunning():
            return
        selectedInterface = self.interfaceDropdown.currentData()
        currentFilter = self.bpfFilterInput.text()
        self.clearRecords()
        self.liveWorker = LiveCaptureWorker(
            selectedInterface, self.dbPath, currentFilter, packetBuffer=self.capturedPackets
        )
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.errorOccurred.connect(
            lambda errorMsg: self.detailsPane.setText(errorMsg)
        )
        self.liveWorker.captureFinished.connect(self.onCaptureFinished)
        self.liveWorker.start()
        self.liveButton.setText("Stop Live Capture")
        self.loadButton.setEnabled(False)
        self.cancelLoadButton.setEnabled(False)

```

```

def clearRecords(self):
    self.dataTable.setRowCount(0)
    self.rowCache.clear()
    self.capturedPackets.clear()
    self.loadedPcapPath = None

```

```

self.detailsPane.clear()
self.labelButton.setEnabled(False)

def savePcapFile(self):
    if not self.capturedPackets and not self.loadedPcapPath:
        self.detailsPane.setText("No captured or loaded packets available to save.")
        return
    savePath, _ = QFileDialog.getSaveFileName(
        self, "Save PCAP", str(self.pcapsDir / "captured.pcap"), "PCAP Files (*.pcap *.pcapng)"
    )
    if not savePath:
        return
    try:
        if self.capturedPackets:
            packetsSnapshot = list(self.capturedPackets)
            wrpcap(savePath, packetsSnapshot)
            self.detailsPane.setText(f"Successfully saved {len(packetsSnapshot)} packet(s) to {savePath}")
        elif self.loadedPcapPath:
            shutil.copyfile(self.loadedPcapPath, savePath)
            self.detailsPane.setText(f"Successfully exported PCAP to {savePath}")
    except Exception as err:
        self.detailsPane.setText(f"Failed to save PCAP: {err}")

def appendTableRow(self, packetData):
    currentRow = self.dataTable.rowCount()
    self.dataTable.insertRow(currentRow)
    self.rowCache.append(packetData)
    columnValues = [
        packetData["role"],
        packetData["source"],
        packetData["destination"],
        packetData["sni"],
        packetData["ja3Hash"],
        packetData["ja4Hash"],
        packetData["ja3Match"],
        packetData["ja4Match"],
    ]
    isClient = packetData["role"] == "Client"
    rowColor = QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20)
    for colIdx, val in enumerate(columnValues):
        item = QTableWidgetItem(str(val))
        item.setFlags(item.flags() & ~Qt.ItemFlag.ItemIsEditable)
        item.setBackground(rowColor)
        self.dataTable.setItem(currentRow, colIdx, item)
    self.dataTable.scrollToBottom()

def displayRowDetails(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        self.labelButton.setEnabled(False)
        return
    rowIndex = selectedRows[0].row()

```

```

if rowIndex < len(self.rowCache):
    entry = self.rowCache[rowIndex]
    isUnknown = entry.get("ja3Match") == "Unknown" or entry.get("ja4Match") == "Unknown"
    self.labelButton.setEnabled(isUnknown)
    detailText = (
        f"Role:      {entry['role']}\n"
        f"SNI:        {entry['sni']}\n"
        f"JA3 Hash:    {entry['ja3Hash']}\n"
        f"JA3 Match:   {entry['ja3Match']}\n"
        f"JA4 Hash:    {entry['ja4Hash']}\n"
        f"JA4 Match:   {entry['ja4Match']}\n"
        f"Raw JA3:     {entry.get('ja3Raw', 'N/A')}"
    )
    self.detailsPane.setText(detailText)

```

```

def labelSelectedFingerprint(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        return
    rowIndex = selectedRows[0].row()
    if rowIndex >= len(self.rowCache):
        return
    entry = self.rowCache[rowIndex]
    if entry.get("ja3Match") != "Unknown" and entry.get("ja4Match") != "Unknown":
        return
    label, accepted = QInputDialog.getText(
        self,
        "Label Fingerprint",
        "Verified name:",
    )
    if not accepted or not label.strip():
        return
    try:
        db = self.getDb()
        pair = (
            ("ja3", "ja3Hash", "ja3Match", 6),
            ("ja4", "ja4Hash", "ja4Match", 7),
        )
        if entry.get("fingerprintKind") in ("ja3s", "ja4s"):
            pair = (
                ("ja3s", "ja3Hash", "ja3Match", 6),
                ("ja4s", "ja4Hash", "ja4Match", 7),
            )
        for kind, hashKey, matchKey, _ in pair:
            if entry.get(matchKey) == "Unknown":
                db.enroll(entry[hashKey], kind, label)
    except (OSError, ValueError) as error:
        self.detailsPane.setText(f"Could not save fingerprint: {error}")
    return
    for matchKey, targetColumn in (("ja3Match", 6), ("ja4Match", 7)):
        if entry.get(matchKey) == "Unknown":
            entry[matchKey] = label.strip()

```

```

newItem = QTableWidgetItem(entry[matchKey])
newItem.setFlags(newItem.flags() & ~Qt.ItemFlag.ItemIsEditable)
isClient = entry["role"] == "Client"
newItem.setBackground(QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20))
self.dataTable.setItem(rowIndex, targetColumn, newItem)
self.displayRowDetails()

```

User prompt: Okay now we need 2 big new things. We need to generate timestamps for each record as they are fed into the gui. My reasoning for generating the timestamp in the gui layer is because a hello message can be more than one packet and it might be harder in the previous layers. So I feel its good enough to generate a timestamp in this layer. Do not show it as a separate field, but maybe it can be displayed on the bottom on the right hand side if you click a record? Also I need a button to open up an analytics panel. This can open up from the right hand side. In it you will see a pie chart of the different matched clients. So it could be like curl, google etc etc. Using the JA3 JA4 Matches. Also I need a line chart which will track the number of messages as time progresses. Also, when you are live capturing, would it be possible to update these graphs every second for example? Use your jurisdiction. For the layout, I want it to possibly be a panel that slides in from the left on top of the existing UI, and the pie chart is on the bottom, the line chart is on the top, or maybe we can have different tabs for them. Do whatever works better. Take your time. I suppose it would be best if we created a new python file for it no?

Response: To introduce a left-side slide-in drawer and record timestamping, you will need to utilize QPropertyAnimation to interpolate widget geometry dynamically and FigureCanvasQTAgg to embed Matplotlib graphics into the PyQt6 window.

Install matplotlib in your environment before running this.

analytics.py

```

from PyQt6.QtWidgets import QWidget, QVBoxLayout
from PyQt6.QtCore import QPropertyAnimation, QRect, QEasingCurve, QTimer
from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg
from matplotlib.figure import Figure
from collections import Counter

class AnalyticsPanel(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setStyleSheet("background-color: #f8f9fa; border-right: 1px solid #ccc;")

        self.layout = QVBoxLayout(self)

        self.figure = Figure(figsize=(4, 6))
        self.canvas = FigureCanvasQTAgg(self.figure)
        self.layout.addWidget(self.canvas)

        self.lineAx = self.figure.add_subplot(211)
        self.pieAx = self.figure.add_subplot(212)

        self.isOpen = False

```

```

self.animation = QPropertyAnimation(self, b"geometry")
self.animation.setDuration(350)
self.animation.setEasingCurve(QEasingCurve.Type.OutCubic)

self.updateTimer = QTimer(self)
self.updateTimer.timeout.connect(self.refreshCharts)

self.rowCacheRef = []
self.setGeometry(-400, 0, 400, 650)

def setRowCache(self, rowCache):
    self.rowCacheRef = rowCache

def togglePanel(self):
    parentHeight = self.parent().height() if self.parent() else 650
    panelWidth = 400

    if not self.isOpen:
        self.animation.setStartValue(QRect(-panelWidth, 0, panelWidth, parentHeight))
        self.animation.setEndValue(QRect(0, 0, panelWidth, parentHeight))
        self.isOpen = True
    else:
        self.animation.setStartValue(QRect(0, 0, panelWidth, parentHeight))
        self.animation.setEndValue(QRect(-panelWidth, 0, panelWidth, parentHeight))
        self.isOpen = False

    self.animation.start()

def startLiveUpdates(self):
    self.updateTimer.start(1000)

def stopLiveUpdates(self):
    self.updateTimer.stop()
    self.refreshCharts()

def refreshCharts(self):
    self.pieAx.clear()
    self.lineAx.clear()

    if not self.rowCacheRef:
        self.canvas.draw()
        return

    matches = []
    timeBuckets = Counter()

    for row in self.rowCacheRef:
        match = row.get("ja3Match", "Unknown")
        if match == "Unknown":
            match = row.get("ja4Match", "Unknown")
        matches.append(match)

```

```

capTime = row.get("captureTime")
if capTime:
    bucket = capTime[:19]
    timeBuckets[bucket] += 1

pieCounts = Counter(matches)
labels = list(pieCounts.keys())
sizes = list(pieCounts.values())

if sizes:
    self.pieAx.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
    self.pieAx.set_title("Client Matches")

if timeBuckets:
    sortedTimes = sorted(timeBuckets.keys())
    counts = [timeBuckets[t] for t in sortedTimes]

    sortedTimes = sortedTimes[-15:]
    counts = counts[-15:]
    xLabels = [t.split(" ")[1] for t in sortedTimes]

    self.lineAx.plot(xLabels, counts, marker='o', color='blue')
    self.lineAx.set_title("Messages Over Time")
    self.lineAx.tick_params(axis='x', rotation=45)

self.figure.tight_layout()
self.canvas.draw()

```

window.py

```

from pathlib import Path
import shutil
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QColor
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
    QWidgetItem, QTableWidgetItem, QFileDialog, QHeaderView,
    QComboBox, QLabel, QTextEdit, QInputDialog, QAbstractItemView,
    QLineEdit
)
from scapy.all import get_working_ifaces, wrpcap
from src.db import FingerprintDB
from workers import PcapWorker, LiveCaptureWorker
from analytics import AnalyticsPanel

class TlsMonitorGui(QMainWindow):
    def __init__(self, pythonEngineDir):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1150, 650)

```

```

self.dbPath = pythonEngineDir / "fingerprints.json"
self.pcapsDir = pythonEngineDir.parent / "pcaps"
self.fingerprintDb = None
self.pcapWorker = None
self.liveWorker = None
self.rowCache = []
self.capturedPackets = []
self.loadedPcapPath = None
mainWidget = QWidget()
self.setCentralWidget(mainWidget)
mainLayout = QVBoxLayout(mainWidget)
controlLayout = QHBoxLayout()
self.loadButton = QPushButton("Load PCAP")
self.loadButton.clicked.connect(self.selectPcapFile)
controlLayout.addWidget(self.loadButton)
self.cancelLoadButton = QPushButton("Cancel Load")
self.cancelLoadButton.clicked.connect(self.cancelPcapLoad)
self.cancelLoadButton.setEnabled(False)
controlLayout.addWidget(self.cancelLoadButton)
self.saveButton = QPushButton("Save PCAP")
self.saveButton.clicked.connect(self.savePcapFile)
controlLayout.addWidget(self.saveButton)
self.clearButton = QPushButton("Clear Records")
self.clearButton.clicked.connect(self.clearRecords)
controlLayout.addWidget(self.clearButton)
self.refreshInterfacesButton = QPushButton("Refresh Interfaces")
self.refreshInterfacesButton.clicked.connect(self.populateInterfaces)
controlLayout.addWidget(self.refreshInterfacesButton)
controlLayout.addWidget(QLabel("Interface:"))
self.interfaceDropdown = QComboBox()
self.populateInterfaces()
controlLayout.addWidget(self.interfaceDropdown)
controlLayout.addWidget(QLabel("BPF Filter:"))
self.bpfFilterInput = QLineEdit("tcp port 443")
controlLayout.addWidget(self.bpfFilterInput)
self.liveButton = QPushButton("Start Live Capture")
self.liveButton.clicked.connect(self.toggleLiveCapture)
controlLayout.addWidget(self.liveButton)
self.labelButton = QPushButton("Label Fingerprint")
self.labelButton.clicked.connect(self.labelSelectedFingerprint)
self.labelButton.setEnabled(False)
controlLayout.addWidget(self.labelButton)

self.analyticsButton = QPushButton("Analytics")
self.analyticsButton.clicked.connect(self.toggleAnalytics)
controlLayout.addWidget(self.analyticsButton)

controlLayout.addStretch()
mainLayout.addLayout(controlLayout)
self.dataTable = QTableWidgetItem(0, 8)
self.dataTable.setHorizontalHeaderLabels([
    "Role", "Source", "Destination", "SNI", "JA3 Hash", "JA4 Hash",

```

```

        "JA3 Match", "JA4 Match"
    ])

    header = self.dataTable.horizontalHeader()
    header.setSectionResizeMode(QHeaderView.ResizeMode.ResizeToContents)
    header.setSectionResizeMode(3, QHeaderView.ResizeMode.Stretch)
    self.dataTable.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows)
    self.dataTable.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
    self.dataTable.itemSelectionChanged.connect(self.displayRowDetails)
    mainLayout.addWidget(self.dataTable)

    detailsLayout = QHBoxLayout()
    self.detailsPane = QTextEdit()
    self.detailsPane.setReadOnly(True)
    self.detailsPane.setFixedHeight(120)
    self.detailsPane.setPlaceholderText("Select any handshake above to view raw metadata and JA3
string.")
    detailsLayout.addWidget(self.detailsPane)

    self.timestampLabel = QLabel("Timestamp: N/A")
    self.timestampLabel.setAlignment(Qt.AlignmentFlag.AlignBottom | Qt.AlignmentFlag.AlignRight)
    detailsLayout.addWidget(self.timestampLabel)

    mainLayout.addLayout(detailsLayout)

    self.analyticsPanel = AnalyticsPanel(self)
    self.analyticsPanel.setRowCache(self.rowCache)

def resizeEvent(self, event):
    super().resizeEvent(event)
    if hasattr(self, 'analyticsPanel'):
        self.analyticsPanel.setFixedHeight(self.height())
        if self.analyticsPanel.isOpen:
            self.analyticsPanel.setGeometry(0, 0, 400, self.height())
        else:
            self.analyticsPanel.setGeometry(-400, 0, 400, self.height())

def toggleAnalytics(self):
    self.analyticsPanel.togglePanel()
    if self.analyticsPanel.isOpen:
        self.analyticsPanel.refreshCharts()
    self.analyticsPanel.raise_()

def populateInterfaces(self):
    self.interfaceDropdown.clear()
    self.interfaceDropdown.addItem("Any (All Interfaces)", "Any")
    try:
        for iface in get_working_ifaces():
            self.interfaceDropdown.addItem(iface.name, iface.network_name)
    except Exception:
        self.interfaceDropdown.addItem("Default", "Default")

```

```

def closeEvent(self, event):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.pcapWorker.wait()
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveWorker.wait()
    event.accept()

def getDb(self):
    if self.fingerprintDb is None:
        self.fingerprintDb = FingerprintDB(str(self.dbPath))
    return self.fingerprintDb

def selectPcapFile(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        return
    filePath, _ = QFileDialog.getOpenFileName(
        self, "Open PCAP", str(self.pcapsDir), "PCAP Files (*.pcap *.pcapng)"
    )
    if filePath:
        self.clearRecords()
        self.loadedPcapPath = filePath
        self.loadButton.setEnabled(False)
        self.liveButton.setEnabled(False)
        self.cancelLoadButton.setEnabled(True)
        self.pcapWorker = PcapWorker(filePath, self.dbPath)
        self.pcapWorker.rowExtracted.connect(self.appendTableRow)
        self.pcapWorker.errorOccurred.connect(lambda err: self.detailsPane.setText(f"Error: {err}"))
        self.pcapWorker.captureFinished.connect(self.onCaptureFinished)
        self.analyticsPanel.refreshCharts()
        self.pcapWorker.start()

def cancelPcapLoad(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.detailsPane.append("\nPCAP loading cancelled.")
        self.analyticsPanel.refreshCharts()

def onCaptureFinished(self, statsSummary):
    self.loadButton.setEnabled(True)
    self.liveButton.setEnabled(True)
    self.cancelLoadButton.setEnabled(False)
    currentText = self.detailsPane.toPlainText()
    self.detailsPane.setText(f"{currentText}\n\nCapture Finished.\n{statsSummary}")
    self.analyticsPanel.refreshCharts()

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)

```

```

        self.analyticsPanel.stopLiveUpdates()
    else:
        if self.pcapWorker and self.pcapWorker.isRunning():
            return
        selectedInterface = self.interfaceDropdown.currentData()
        currentFilter = self.bpfFilterInput.text()
        self.clearRecords()
        self.liveWorker = LiveCaptureWorker(
            selectedInterface, self.dbPath, currentFilter, packetBuffer=self.capturedPackets
        )
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.errorOccurred.connect(
            lambda errorMsg: self.detailsPane.setText(errorMsg)
        )
        self.liveWorker.captureFinished.connect(self.onCaptureFinished)
        self.liveWorker.start()
        self.liveButton.setText("Stop Live Capture")
        self.loadButton.setEnabled(False)
        self.cancelLoadButton.setEnabled(False)
        self.analyticsPanel.startLiveUpdates()

def clearRecords(self):
    self.dataTable.setRowCount(0)
    self.rowCache.clear()
    self.capturedPackets.clear()
    self.loadedPcapPath = None
    self.detailsPane.clear()
    self.timestampLabel.setText("Timestamp: N/A")
    self.labelButton.setEnabled(False)
    self.analyticsPanel.refreshCharts()

def savePcapFile(self):
    if not self.capturedPackets and not self.loadedPcapPath:
        self.detailsPane.setText("No captured or loaded packets available to save.")
        return
    savePath, _ = QFileDialog.getSaveFileName(
        self, "Save PCAP", str(self.pcapsDir / "captured.pcap"), "PCAP Files (*.pcap *.pcapng)"
    )
    if not savePath:
        return
    try:
        if self.capturedPackets:
            packetsSnapshot = list(self.capturedPackets)
            wrpcap(savePath, packetsSnapshot)
            self.detailsPane.setText(f"Successfully saved {len(packetsSnapshot)} packet(s) to {savePath}")
        elif self.loadedPcapPath:
            shutil.copyfile(self.loadedPcapPath, savePath)
            self.detailsPane.setText(f"Successfully exported PCAP to {savePath}")
    except Exception as err:
        self.detailsPane.setText(f"Failed to save PCAP: {err}")

def appendTableRow(self, packetData):

```

```
if "captureTime" not in packetData:
    packetData["captureTime"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
```

```
currentRow = self.dataTable.rowCount()
self.dataTable.insertRow(currentRow)
self.rowCache.append(packetData)
columnValues = [
    packetData["role"],
    packetData["source"],
    packetData["destination"],
    packetData["sni"],
    packetData["ja3Hash"],
    packetData["ja4Hash"],
    packetData["ja3Match"],
    packetData["ja4Match"],
]
isClient = packetData["role"] == "Client"
rowColor = QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20)
for colIdx, val in enumerate(columnValues):
    item = QTableWidgetItem(str(val))
    item.setFlags(item.flags() & ~Qt.ItemFlag.ItemIsEditable)
    item.setBackground(rowColor)
    self.dataTable.setItem(currentRow, colIdx, item)
self.dataTable.scrollToBottom()
```

```
def displayRowDetails(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        self.labelButton.setEnabled(False)
        return
    rowIndex = selectedRows[0].row()
    if rowIndex < len(self.rowCache):
        entry = self.rowCache[rowIndex]
        isUnknown = entry.get("ja3Match") == "Unknown" or entry.get("ja4Match") == "Unknown"
        self.labelButton.setEnabled(isUnknown)
        detailText = (
            f"Role:      {entry['role']}\n"
            f"SNI:       {entry['sni']}\n"
            f"JA3 Hash:   {entry['ja3Hash']}\n"
            f"JA3 Match:  {entry['ja3Match']}\n"
            f"JA4 Hash:   {entry['ja4Hash']}\n"
            f"JA4 Match:  {entry['ja4Match']}\n"
            f"Raw JA3:    {entry.get('ja3Raw', 'N/A')}"
        )
        self.detailsPane.setText(detailText)

        capTime = entry.get("captureTime", "N/A")
        self.timestampLabel.setText(f"Timestamp:\n{capTime}")
```

```
def labelSelectedFingerprint(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
```

```

        return
    rowIndex = selectedRows[0].row()
    if rowIndex >= len(self.rowCache):
        return
    entry = self.rowCache[rowIndex]
    if entry.get("ja3Match") != "Unknown" and entry.get("ja4Match") != "Unknown":
        return
    label, accepted = QInputDialog.getText(
        self,
        "Label Fingerprint",
        "Verified name:",
    )
    if not accepted or not label.strip():
        return
    try:
        db = self.getDb()
        pair = (
            ("ja3", "ja3Hash", "ja3Match", 6),
            ("ja4", "ja4Hash", "ja4Match", 7),
        )
        if entry.get("fingerprintKind") in ("ja3s", "ja4s"):
            pair = (
                ("ja3s", "ja3Hash", "ja3Match", 6),
                ("ja4s", "ja4Hash", "ja4Match", 7),
            )
        for kind, hashKey, matchKey, _ in pair:
            if entry.get(matchKey) == "Unknown":
                db.enroll(entry[hashKey], kind, label)
    except (OSError, ValueError) as error:
        self.detailsPane.setText(f"Could not save fingerprint: {error}")
        return
    for matchKey, targetColumn in (("ja3Match", 6), ("ja4Match", 7)):
        if entry.get(matchKey) == "Unknown":
            entry[matchKey] = label.strip()
            newItem = QTableWidgetItem(entry[matchKey])
            newItem.setFlags(newItem.flags() & ~Qt.ItemFlag.ItemIsEditable)
            isClient = entry["role"] == "Client"
            newItem.setBackground(QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20))
            self.dataTable.setItem(rowIndex, targetColumn, newItem)
    self.displayRowDetails()
    self.analyticsPanel.refreshCharts()

```

User prompt: problems: The graph looks like shit. Currently its just a white background. Can it somehow be system specific? Like my system is dark and the main window is dark for me, however for my friend whose system is light, the main window is white. So I want a dark background on a dark system and a light background on a light system (If possible) Also, the line graph is stupid? The X axis is not time consistent. It looks like you are adding a X axis value on every new timestamp being added, which makes the time difference between successive timestamps redundant.

Response: To adapt the Matplotlib canvas dynamically, you can extract the active OS-inherited window and

text colors directly from the PyQt6 QPalette. To fix the X-axis time consistency without crashing the application on large PCAP files spanning hours, you can parse the timestamps into native datetime objects. Matplotlib will space them proportionally by default; we just need to strategically inject a value of 0 at the edges of time gaps so the line drops visually during quiet periods.

analytics.py

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout
from PyQt6.QtCore import QPropertyAnimation, QRect, QEasingCurve, QTimer
from PyQt6.QtGui import QPalette
from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg
from matplotlib.figure import Figure
import matplotlib.dates as mdates
from collections import Counter
from datetime import datetime, timedelta

class AnalyticsPanel(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        # Inherit the main window's background rather than hardcoding white
        self.setStyleSheet("border-right: 1px solid #ccc;")

        self.layout = QVBoxLayout(self)

        self.figure = Figure(figsize=(4, 6))
        self.canvas = FigureCanvasQTAgg(self.figure)
        self.layout.addWidget(self.canvas)

        self.lineAx = self.figure.add_subplot(211)
        self.pieAx = self.figure.add_subplot(212)

        self.isOpen = False
        self.animation = QPropertyAnimation(self, b"geometry")
        self.animation.setDuration(350)
        self.animation.setEasingCurve(QEasingCurve.Type.OutCubic)

        self.updateTimer = QTimer(self)
        self.updateTimer.timeout.connect(self.refreshCharts)

        self.rowCacheRef = []
        self.setGeometry(-400, 0, 400, 650)

    def setRowCache(self, rowCache):
        self.rowCacheRef = rowCache

    def togglePanel(self):
        parentHeight = self.parent().height() if self.parent() else 650
        panelWidth = 400

        if not self.isOpen:
            self.animation.setStartValue(QRect(-panelWidth, 0, panelWidth, parentHeight))
            self.animation.setEndValue(QRect(0, 0, panelWidth, parentHeight))
```

```

        self.isOpen = True
    else:
        self.animation.setStartValue(QRect(0, 0, panelWidth, parentHeight))
        self.animation.setEndValue(QRect(-panelWidth, 0, panelWidth, parentHeight))
        self.isOpen = False

    self.animation.start()

def startLiveUpdates(self):
    self.updateTimer.start(1000)

def stopLiveUpdates(self):
    self.updateTimer.stop()
    self.refreshCharts()

def refreshCharts(self):
    self.pieAx.clear()
    self.lineAx.clear()

    # Query the active OS-inherited Qt palette to style Matplotlib components dynamically
    bgColor = self.palette().color(QPalette.ColorRole.Window).name()
    textColor = self.palette().color(QPalette.ColorRole.WindowText).name()

    self.figure.patch.set_facecolor(bgColor)
    for ax in (self.pieAx, self.lineAx):
        ax.set_facecolor(bgColor)
        ax.tick_params(colors=textColor)
        for spine in ax.spines.values():
            spine.set_edgecolor(textColor)
        ax.title.set_color(textColor)

    if not self.rowCacheRef:
        self.canvas.draw()
        return

    matches = []
    timeBuckets = Counter()

    for row in self.rowCacheRef:
        match = row.get("ja3Match", "Unknown")
        if match == "Unknown":
            match = row.get("ja4Match", "Unknown")
        matches.append(match)

        capTime = row.get("captureTime")
        if capTime:
            bucket = capTime[:19]
            timeBuckets[bucket] += 1

    pieCounts = Counter(matches)
    labels = list(pieCounts.keys())
    sizes = list(pieCounts.values())

```

```

if sizes:
    self.pieAx.pie(
        sizes, labels=labels, autopct='%1.1f%%', startangle=90,
        textprops={'color': textColor}
    )
    self.pieAx.set_title("Client Matches")

if timeBuckets:
    parsedTimes = {datetime.strptime(k, "%Y-%m-%d %H:%M:%S"): v for k, v in timeBuckets.items()}
    sortedDates = sorted(parsedTimes.keys())

    xData = []
    yData = []

    # Map timestamps to proper datetime objects for proportional X-axis spacing
    for i in range(len(sortedDates)):
        currentDate = sortedDates[i]
        xData.append(currentDate)
        yData.append(parsedTimes[currentDate])

    # Force the line to drop to 0 during gaps rather than connecting points directly over dead time
    if i < len(sortedDates) - 1:
        nextDate = sortedDates[i + 1]
        gapSeconds = (nextDate - currentDate).total_seconds()
        if gapSeconds > 1:
            xData.append(currentDate + timedelta(seconds=1))
            yData.append(0)
        if gapSeconds > 2:
            xData.append(nextDate - timedelta(seconds=1))
            yData.append(0)

    self.lineAx.plot(xData, yData, marker='o', color='tab:blue')
    self.lineAx.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M:%S'))
    self.lineAx.set_title("Messages Over Time")
    self.lineAx.tick_params(axis='x', rotation=45)

self.figure.tight_layout()
self.canvas.draw()

```

User prompt: New problem: I want the actual window to be squeezed to the right when I press the analytics button, because that is hiding important information. So it will kinda be like a sidebar.

Response: To transform the floating overlay into an integrated collapsible sidebar that pushes and squeezes the main content, the central widget layout has been restructured into an outer QHBoxLayout. The animation now targets an explicit panelWidth property rather than floating geometry coordinates.

analytics.py

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout
from PyQt6.QtCore import pyqtProperty, QPropertyAnimation, QEasingCurve, QTimer
from PyQt6.QtGui import QPalette
from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg
from matplotlib.figure import Figure
import matplotlib.dates as mdates
from collections import Counter
from datetime import datetime, timedelta
```

```
class AnalyticsPanel(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setStyleSheet("border-right: 1px solid #ccc;")

        self.panelLayout = QVBoxLayout(self)
        self.panelLayout.setContentsMargins(0, 0, 0, 0)

        self.figure = Figure(figsize=(4, 6))
        self.canvas = FigureCanvasQTAgg(self.figure)
        self.panelLayout.addWidget(self.canvas)

        self.lineAx = self.figure.add_subplot(211)
        self.pieAx = self.figure.add_subplot(212)

        self.targetWidth = 380
        self.isOpen = False

        self.setFixedWidth(0)
        self.setVisible(False)
        self.canvas.setVisible(False)

        self.animation = QPropertyAnimation(self, b"panelWidth")
        self.animation.setDuration(250)
        self.animation.setEasingCurve(QEasingCurve.Type.OutCubic)
        self.animation.finished.connect(self.onAnimationFinished)

        self.updateTimer = QTimer(self)
        self.updateTimer.timeout.connect(self.refreshCharts)

        self.rowCacheRef = []

    def getPanelWidth(self):
        return self.width()

    def setPanelWidth(self, width):
        self.setFixedWidth(width)
        if width <= 10:
            self.canvas.setVisible(False)
        else:
            self.canvas.setVisible(True)

    panelWidth = pyqtProperty(int, getPanelWidth, setPanelWidth)
```

```

def setRowCache(self, rowCache):
    self.rowCacheRef = rowCache

def togglePanel(self):
    if not self.isOpen:
        self.setVisible(True)
        self.animation.stop()
        self.animation.setStartValue(self.width())
        self.animation.setEndValue(self.targetWidth)
        self.isOpen = True
        self.animation.start()
    else:
        self.animation.stop()
        self.animation.setStartValue(self.width())
        self.animation.setEndValue(0)
        self.isOpen = False
        self.animation.start()

def onAnimationFinished(self):
    if not self.isOpen:
        self.setVisible(False)
    else:
        self.refreshCharts()

def startLiveUpdates(self):
    self.updateTimer.start(1000)

def stopLiveUpdates(self):
    self.updateTimer.stop()
    if self.isOpen:
        self.refreshCharts()

def refreshCharts(self):
    if not self.isOpen or self.width() < 100:
        return

    self.pieAx.clear()
    self.lineAx.clear()

    bgColor = self.palette().color(QPalette.ColorRole.Window).name()
    textColor = self.palette().color(QPalette.ColorRole.WindowText).name()

    self.figure.patch.set_facecolor(bgColor)
    for ax in (self.pieAx, self.lineAx):
        ax.set_facecolor(bgColor)
        ax.tick_params(colors=textColor)
        for spine in ax.spines.values():
            spine.set_edgecolor(textColor)
        ax.title.set_color(textColor)

    if not self.rowCacheRef:

```

```

self.canvas.draw()
return

matches = []
timeBuckets = Counter()

for row in self.rowCacheRef:
    match = row.get("ja3Match", "Unknown")
    if match == "Unknown":
        match = row.get("ja4Match", "Unknown")
    matches.append(match)

    capTime = row.get("captureTime")
    if capTime:
        bucket = capTime[:19]
        timeBuckets[bucket] += 1

pieCounts = Counter(matches)
labels = list(pieCounts.keys())
sizes = list(pieCounts.values())

if sizes:
    self.pieAx.pie(
        sizes, labels=labels, autopct='%1.1f%%', startangle=90,
        textprops={'color': textColor}
    )
    self.pieAx.set_title("Client Matches")

if timeBuckets:
    parsedTimes = {datetime.strptime(k, "%Y-%m-%d %H:%M:%S"): v for k, v in timeBuckets.items()}
    sortedDates = sorted(parsedTimes.keys())

    xData = []
    yData = []

    for i in range(len(sortedDates)):
        currentDate = sortedDates[i]
        xData.append(currentDate)
        yData.append(parsedTimes[currentDate])

        if i < len(sortedDates) - 1:
            nextDate = sortedDates[i + 1]
            gapSeconds = (nextDate - currentDate).total_seconds()
            if gapSeconds > 1:
                xData.append(currentDate + timedelta(seconds=1))
                yData.append(0)
            if gapSeconds > 2:
                xData.append(nextDate - timedelta(seconds=1))
                yData.append(0)

    self.lineAx.plot(xData, yData, marker='o', color='tab:blue')
    self.lineAx.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M:%S'))

```

```

self.lineAx.set_title("Messages Over Time")
self.lineAx.tick_params(axis='x', rotation=45)

self.figure.tight_layout()
self.canvas.draw()

```

window.py

```

from pathlib import Path
import shutil
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QColor
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
    QTableWidgetItem, QTableWidgetItem, QFileDialog, QHeaderView,
    QComboBox, QLabel, QTextEdit, QInputDialog, QAbstractItemView,
    QLineEdit
)
from scapy.all import get_working_ifaces, wrpcap
from src.db import FingerprintDB
from workers import PcapWorker, LiveCaptureWorker
from analytics import AnalyticsPanel

class TlsMonitorGui(QMainWindow):
    def __init__(self, pythonEngineDir):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1150, 650)
        self.dbPath = pythonEngineDir / "fingerprints.json"
        self.pcapsDir = pythonEngineDir.parent / "pcaps"
        self.fingerprintDb = None
        self.pcapWorker = None
        self.liveWorker = None
        self.rowCache = []
        self.capturedPackets = []
        self.loadedPcapPath = None

        centralContainer = QWidget()
        self.setCentralWidget(centralContainer)
        rootLayout = QHBoxLayout(centralContainer)
        rootLayout.setContentsMargins(0, 0, 0, 0)
        rootLayout.setSpacing(0)

        self.analyticsPanel = AnalyticsPanel(self)
        self.analyticsPanel.setRowCache(self.rowCache)
        rootLayout.addWidget(self.analyticsPanel)

        contentWidget = QWidget()
        mainLayout = QVBoxLayout(contentWidget)
        mainLayout.setContentsMargins(8, 8, 8, 8)

```

```

rootLayout.addWidget(contentWidget)

controlLayout = QHBoxLayout()
self.loadButton = QPushButton("Load PCAP")
self.loadButton.clicked.connect(self.selectPcapFile)
controlLayout.addWidget(self.loadButton)
self.cancelLoadButton = QPushButton("Cancel Load")
self.cancelLoadButton.clicked.connect(self.cancelPcapLoad)
self.cancelLoadButton.setEnabled(False)
controlLayout.addWidget(self.cancelLoadButton)
self.saveButton = QPushButton("Save PCAP")
self.saveButton.clicked.connect(self.savePcapFile)
controlLayout.addWidget(self.saveButton)
self.clearButton = QPushButton("Clear Records")
self.clearButton.clicked.connect(self.clearRecords)
controlLayout.addWidget(self.clearButton)
self.refreshInterfacesButton = QPushButton("Refresh Interfaces")
self.refreshInterfacesButton.clicked.connect(self.populateInterfaces)
controlLayout.addWidget(self.refreshInterfacesButton)
controlLayout.addWidget(QLabel("Interface:"))
self.interfaceDropdown = QComboBox()
self.populateInterfaces()
controlLayout.addWidget(self.interfaceDropdown)
controlLayout.addWidget(QLabel("BPF Filter:"))
self.bpfFilterInput = QLineEdit("tcp port 443")
controlLayout.addWidget(self.bpfFilterInput)
self.liveButton = QPushButton("Start Live Capture")
self.liveButton.clicked.connect(self.toggleLiveCapture)
controlLayout.addWidget(self.liveButton)
self.labelButton = QPushButton("Label Fingerprint")
self.labelButton.clicked.connect(self.labelSelectedFingerprint)
self.labelButton.setEnabled(False)
controlLayout.addWidget(self.labelButton)

self.analyticsButton = QPushButton("Analytics")
self.analyticsButton.clicked.connect(self.toggleAnalytics)
controlLayout.addWidget(self.analyticsButton)

controlLayout.addStretch()
mainLayout.addLayout(controlLayout)

self.dataTable = QTableWidget(0, 8)
self.dataTable.setHorizontalHeaderLabels([
    "Role", "Source", "Destination", "SN1", "JA3 Hash", "JA4 Hash",
    "JA3 Match", "JA4 Match"
])

header = self.dataTable.horizontalHeader()
header.setSectionResizeMode(QHeaderView.ResizeMode.ResizeToContents)
header.setSectionResizeMode(3, QHeaderView.ResizeMode.Stretch)
self.dataTable.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows)
self.dataTable.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)

```

```

self.dataTable.itemSelectionChanged.connect(self.displayRowDetails)
mainLayout.addWidget(self.dataTable)

detailsLayout = QHBoxLayout()
self.detailsPane = QTextEdit()
self.detailsPane.setReadOnly(True)
self.detailsPane.setFixedHeight(120)
self.detailsPane.setPlaceholderText("Select any handshake above to view raw metadata and JA3
string.")
detailsLayout.addWidget(self.detailsPane)

self.timestampLabel = QLabel("Timestamp: N/A")
self.timestampLabel.setAlignment(Qt.AlignmentFlag.AlignBottom | Qt.AlignmentFlag.AlignRight)
detailsLayout.addWidget(self.timestampLabel)

mainLayout.addLayout(detailsLayout)

def toggleAnalytics(self):
    self.analyticsPanel.togglePanel()

def populateInterfaces(self):
    self.interfaceDropdown.clear()
    self.interfaceDropdown.addItem("Any (All Interfaces)", "Any")
    try:
        for iface in get_working_ifaces():
            self.interfaceDropdown.addItem(iface.name, iface.network_name)
    except Exception:
        self.interfaceDropdown.addItem("Default", "Default")

def closeEvent(self, event):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.pcapWorker.wait()
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveWorker.wait()
    event.accept()

def getDb(self):
    if self.fingerprintDb is None:
        self.fingerprintDb = FingerprintDB(str(self.dbPath))
    return self.fingerprintDb

def selectPcapFile(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        return
    filePath, _ = QFileDialog.getOpenFileName(
        self, "Open PCAP", str(self.pcapsDir), "PCAP Files (*.pcap *.pcapng)"
    )
    if filePath:
        self.clearRecords()
        self.loadedPcapPath = filePath

```

```

self.loadButton.setEnabled(False)
self.liveButton.setEnabled(False)
self.cancelLoadButton.setEnabled(True)
self.pcapWorker = PcapWorker(filePath, self.dbPath)
self.pcapWorker.rowExtracted.connect(self.appendTableRow)
self.pcapWorker.errorOccurred.connect(lambda err: self.detailsPane.setText(f"Error: {err}"))
self.pcapWorker.captureFinished.connect(self.onCaptureFinished)
self.analyticsPanel.refreshCharts()
self.pcapWorker.start()

```

```

def cancelPcapLoad(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.detailsPane.append("\nPCAP loading cancelled.")
        self.analyticsPanel.refreshCharts()

```

```

def onCaptureFinished(self, statsSummary):
    self.loadButton.setEnabled(True)
    self.liveButton.setEnabled(True)
    self.cancelLoadButton.setEnabled(False)
    currentText = self.detailsPane.toPlainText()
    self.detailsPane.setText(f"{currentText}\n\nCapture Finished.\n{statsSummary}")
    self.analyticsPanel.refreshCharts()

```

```

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
        self.analyticsPanel.stopLiveUpdates()
    else:
        if self.pcapWorker and self.pcapWorker.isRunning():
            return
        selectedInterface = self.interfaceDropdown.currentData()
        currentFilter = self.bpfFilterInput.text()
        self.clearRecords()
        self.liveWorker = LiveCaptureWorker(
            selectedInterface, self.dbPath, currentFilter, packetBuffer=self.capturedPackets
        )
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.errorOccurred.connect(
            lambda errorMsg: self.detailsPane.setText(errorMsg)
        )
        self.liveWorker.captureFinished.connect(self.onCaptureFinished)
        self.liveWorker.start()
        self.liveButton.setText("Stop Live Capture")
        self.loadButton.setEnabled(False)
        self.cancelLoadButton.setEnabled(False)
        self.analyticsPanel.startLiveUpdates()

```

```

def clearRecords(self):
    self.dataTable.setRowCount(0)

```

```

self.rowCache.clear()
self.capturedPackets.clear()
self.loadedPcapPath = None
self.detailsPane.clear()
self.timestampLabel.setText("Timestamp: N/A")
self.labelButton.setEnabled(False)
self.analyticsPanel.refreshCharts()

```

```

def savePcapFile(self):
    if not self.capturedPackets and not self.loadedPcapPath:
        self.detailsPane.setText("No captured or loaded packets available to save.")
        return
    savePath, _ = QFileDialog.getSaveFileName(
        self, "Save PCAP", str(self.pcapsDir / "captured.pcap"), "PCAP Files (*.pcap *.pcapng)"
    )
    if not savePath:
        return
    try:
        if self.capturedPackets:
            packetsSnapshot = list(self.capturedPackets)
            wrpcap(savePath, packetsSnapshot)
            self.detailsPane.setText(f"Successfully saved {len(packetsSnapshot)} packet(s) to {savePath}")
        elif self.loadedPcapPath:
            shutil.copyfile(self.loadedPcapPath, savePath)
            self.detailsPane.setText(f"Successfully exported PCAP to {savePath}")
    except Exception as err:
        self.detailsPane.setText(f"Failed to save PCAP: {err}")

```

```

def appendTableRow(self, packetData):
    if "captureTime" not in packetData:
        packetData["captureTime"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]

```

```

currentRow = self.dataTable.rowCount()
self.dataTable.insertRow(currentRow)
self.rowCache.append(packetData)
columnValues = [
    packetData["role"],
    packetData["source"],
    packetData["destination"],
    packetData["sni"],
    packetData["ja3Hash"],
    packetData["ja4Hash"],
    packetData["ja3Match"],
    packetData["ja4Match"],
]

```

```

isClient = packetData["role"] == "Client"
rowColor = QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20)
for colIdx, val in enumerate(columnValues):
    item = QTableWidgetItem(str(val))
    item.setFlags(item.flags() & ~Qt.ItemFlag.ItemIsEditable)
    item.setBackground(rowColor)
    self.dataTable.setItem(currentRow, colIdx, item)

```

```

self.dataTable.scrollToBottom()

def displayRowDetails(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        self.labelButton.setEnabled(False)
        return
    rowIndex = selectedRows[0].row()
    if rowIndex < len(self.rowCache):
        entry = self.rowCache[rowIndex]
        isUnknown = entry.get("ja3Match") == "Unknown" or entry.get("ja4Match") == "Unknown"
        self.labelButton.setEnabled(isUnknown)
        detailText = (
            f"Role: {entry['role']}\n"
            f"SNI: {entry['sni']}\n"
            f"JA3 Hash: {entry['ja3Hash']}\n"
            f"JA3 Match: {entry['ja3Match']}\n"
            f"JA4 Hash: {entry['ja4Hash']}\n"
            f"JA4 Match: {entry['ja4Match']}\n"
            f"Raw JA3: {entry.get('ja3Raw', 'N/A')}"
        )
        self.detailsPane.setText(detailText)

        capTime = entry.get("captureTime", "N/A")
        self.timestampLabel.setText(f"Timestamp:\n{capTime}")

def labelSelectedFingerprint(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        return
    rowIndex = selectedRows[0].row()
    if rowIndex >= len(self.rowCache):
        return
    entry = self.rowCache[rowIndex]
    if entry.get("ja3Match") != "Unknown" and entry.get("ja4Match") != "Unknown":
        return
    label, accepted = QDialog.getText(
        self,
        "Label Fingerprint",
        "Verified name:",
    )
    if not accepted or not label.strip():
        return
    try:
        db = self.getDb()
        pair = (
            ("ja3", "ja3Hash", "ja3Match", 6),
            ("ja4", "ja4Hash", "ja4Match", 7),
        )
        if entry.get("fingerprintKind") in ("ja3s", "ja4s"):
            pair = (
                ("ja3s", "ja3Hash", "ja3Match", 6),

```

```

        ("ja4s", "ja4Hash", "ja4Match", 7),
    )
    for kind, hashKey, matchKey, _ in pair:
        if entry.get(matchKey) == "Unknown":
            db.enroll(entry[hashKey], kind, label)
    except (OSError, ValueError) as error:
        self.detailsPane.setText(f"Could not save fingerprint: {error}")
    return
    for matchKey, targetColumn in (("ja3Match", 6), ("ja4Match", 7)):
        if entry.get(matchKey) == "Unknown":
            entry[matchKey] = label.strip()
            newItem = QTableWidgetItem(entry[matchKey])
            newItem.setFlags(newItem.flags() & ~Qt.ItemFlag.ItemIsEditable)
            isClient = entry["role"] == "Client"
            newItem.setBackground(QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20))
            self.dataTable.setItem(rowIndex, targetColumn, newItem)
    self.displayRowDetails()
    self.analyticsPanel.refreshCharts()

```

User prompt: The reason why I am getting unknown in every matched client is because I have none of this setup, as I am on windows. I am now gonna paste in the readme of the repo, which is for linux. is there some way I can make this work for windows somehow? Like any realm of possibility? Because I do not have a linux boot.

CS 331 Computer Networks Team T014 - Project 11: TLS Fingerprinting Team Members: Ujas Devang Shah 24110376 Satyakam Mishra 24110318 Pranjal Goyal 24110274 Harshal Kundanlal Rana 24110131 Divyansh Sharma 24110113 D A S K R Manogna 24110097

About the project TLSFP Engine is a high-performance passive network telemetry tool written in C++17 designed to inspect, reassemble, and fingerprint encrypted traffic without payload decryption. While modern traffic is encrypted under TLS 1.2 and TLS 1.3, the initial cryptographic negotiation in ClientHello and ServerHello packets remains observable in plaintext on the wire. By extracting cipher suites, extensions, elliptic curves, and ALPN parameters, the engine constructs deterministic behavioral signatures for connecting applications. It implements both Salesforce JA3/JA3S and next-generation FoxIO JA4/JA4S specifications, incorporating RFC 8701 GREASE normalization and lexical parameter sorting. The pipeline uses libpcap for zero-copy frame peeling alongside RFC 1982-compliant TCP stream reassembly to handle packet fragmentation and TLS 1.3 middlebox records seamlessly. Extracted hashes are matched against an in-memory Redis database to distinguish tools like cURL, Python requests, Google Chrome, and Mozilla Firefox purely from their handshake. The engine operates across both live interfaces and offline PCAPs, featuring dedicated quiet benchmark (-q) and verbose dissection (-v) modes for performance profiling and deep protocol analysis.

Quick start The project is developed and tested on Ubuntu 24.04 and Ubuntu under WSL. Redis runs locally on 127.0.0.1:6379; the repository JSON files seed the Redis database when either implementation starts. Install system dependencies sudo apt update sudo apt install -y build-essential cmake libpcap-dev libssl-dev redis-server python3-venv sudo systemctl enable --now redis-server Redis database setup Redis runs locally at 127.0.0.1:6379 using database 0. The Redis data itself is runtime state and is not stored in GitHub. The fingerprint catalog is committed to the repository in these files: code/db/seed_fingerprints.json: curated JA3, JA3S, JA4, and JA4S records code/python/fingerprints.json: public and legacy JA3 records No manual database import is required. When the C++ engine or Python application starts, it connects to Redis and automatically seeds the committed records. A fresh clone therefore gets the same catalog after Redis is installed and running. Verify that Redis is available with: redis-cli ping Expected output: PONG If Redis is not running, start it with: sudo systemctl start redis-server The application can use its JSON fallback when Redis is unavailable, but Redis is required for the shared JA3/JA3S/JA4/JA4S catalog, persistent analyst enrollments, and consistent behavior between the C++ engine and Python GUI. Build the C++ engine Run

these commands from the repository root: `cmake -S code/cpp -B build cmake --build build --parallel` Analyze a bundled PCAP without root privileges: `./build/tlsfp_engine -r code/pcaps/test_ja3.pcap` Use `-a` to interactively label an unknown fingerprint after verifying its source: `./build/tlsfp_engine -r code/pcaps/test_ja3.pcap -a` Live capture requires root or suitable packet-capture capabilities: `sudo ./build/tlsfp_engine -i lo -f "tcp port 443"` Run the Python GUI Create the Python environment and install the dependencies: `python3 -m venv .venv source .venv/bin/activate python -m pip install -r code/python/requirements.txt python code/gui/main.py` Click Load PCAP File and select a file under `code/pcaps`. Select an unknown row and use Label Selected Fingerprint only after confirming the client that generated the traffic. Run Python tests source `.venv/bin/activate python -m pytest code/python/tests -q` Import an approved external catalog External datasets must be reviewed for licensing and provenance before import. Use a CSV with `kind,hash,name,role,version,os,category,source,notes` columns, or grouped JSON using the same `ja3`, `ja3s`, `ja4`, and `ja4s` sections as `code/db/seed_fingerprints.json`. source `.venv/bin/activate python code/db/import_fingerprints.py path/to/approved_catalog.csv` Existing curated records are preserved. Use `--overwrite` only when the external record has been verified to be more authoritative.

Response: You can absolutely make this work on Windows without setting up a dual-boot system. Since your academic coursework already involves C, C++, and Python, you can either use Microsoft's virtualized subsystem or adapt the build environment natively.

Method 1: Windows Subsystem for Linux (WSL) The project documentation states it is explicitly tested on "Ubuntu under WSL". WSL is not a separate boot; it is a Linux kernel that runs directly inside your current OS alongside your translucent window modifications and classic menus.

- Open PowerShell as Administrator and run `wsl --install`.
- Restart your machine and open the new Ubuntu terminal app.
- Execute the `sudo apt` commands from the documentation to install `build-essential`, `cmake`, `libpcap-dev`, `libssl-dev`, and `redis-server`.
- Start the database with `sudo systemctl start redis-server`.

Method 2: Native Windows Workarounds If you prefer to run the Python GUI natively in Windows, you must install Windows equivalents for the Linux dependencies.

- **Database:** Redis is required to supply the shared fingerprint catalog, but it lacks native Windows support. Install Memurai, a Redis-compatible datastore for Windows that automatically binds to `127.0.0.1:6379`. This will automatically seed the JSON files into database 0 when the GUI starts, resolving your "Unknown" issue.
- **Packet Capture:** Windows does not use `libpcap-dev`. Install Npcap (ensure "WinPcap API-compatible mode" is checked during installation) so Scapy can monitor your live network interfaces.
- **C++ Engine:** Compiling the C++ telemetry tool natively requires swapping `libpcap-dev` for the Npcap SDK and using a tool like `vcpkg` to link OpenSSL.
- **Python Setup:** Open Command Prompt, run `python -m venv .venv`, activate it, and install the requirements.

Which approach would you prefer to tackle first?

User prompt: if I do the native windows workaround, please tell me I wouldn't have to change the code and

shit would work please

Response: For the Python GUI, you will not have to change a single line of code, but compiling the C++ engine natively will require some surgery.

- **The Python GUI (No Changes):** The codebase uses pathlib for directory navigation, meaning Windows backslashes and Linux forward slashes are handled automatically. Scapy will inherently recognize Npcap as the packet driver. You can run `python main.py` exactly as written.
- **The Database (No Changes):** Memurai operates as a drop-in Windows substitute for Redis. It binds to the identical `127.0.0.1:6379` port and uses the exact same protocol. Your Python DB worker will connect to it without realizing it is not running on Linux.
- **The C++ Engine (Requires Changes):** This is where native Windows gets messy. C++ network telemetry tools built for Ubuntu rely heavily on POSIX networking headers (e.g., `<sys/socket.h>`, `<arpa/inet.h>`). To compile `tlsfp_engine.cpp` on Windows, you will need to manually map these to Windows Sockets (`<winsock2.h>`, `<ws2tcpip.h>`). Furthermore, you will have to rewrite parts of the `CMakeLists.txt` file so CMake knows how to locate the Npcap SDK instead of Linux's `libpcap-dev`.

If your current goal is strictly to work on the GUI layer and capture live traffic using the Python Scapy workers, you can completely ignore the C++ engine for now. Just install Memurai and Npcap, boot up your virtual environment, and the application will run.

User prompt: I do have Ubuntu as WSL installed. is that easier? if it is, please guide me from stage 1 thoroughly how to get my app working

Response: Using WSL is significantly easier because it entirely bypasses the headache of translating POSIX networking headers and manually configuring Npcap for the C++ engine. Assuming you are on a relatively modern Windows build, WSLg will also automatically forward the PyQt6 graphical window right to your desktop.

Step 1: System Update Open your Ubuntu terminal and ensure your package lists are up to date.

- `sudo apt update`

Step 2: Install Core Dependencies Grab the C++ build tools, packet capture libraries, OpenSSL, the Redis database, and the Python virtual environment package.

- `sudo apt install -y build-essential cmake libpcap-dev libssl-dev redis-server python3-venv`

Step 3: Ignite the Database Start the Redis server. This is the critical component you were missing on Windows that caused all the client fingerprints to register as "Unknown."

- `sudo systemctl enable --now redis-server`
- *Fallback:* If WSL complains about systemctl, use `sudo service redis-server start` instead.
- Type `redis-cli ping`. If the terminal returns PONG, your database is actively running on `127.0.0.1:6379`.

Step 4: Configure the Python Environment Use the terminal to navigate (`cd`) into the root directory of your TLS fingerprinting project. Once there, create and activate a clean environment, then install the required

packages.

- `python3 -m venv .venv`
- `source .venv/bin/activate`
- `python -m pip install -r code/python/requirements.txt`
- *Note:* Since we just added Matplotlib for your new analytics panel, make sure it is installed by running `python -m pip install matplotlib` if it isn't already included in the text file.

Step 5: Launch the Application With the virtual environment active and Redis running in the background, you can now start the tool.

- `python code/gui/main.py`

When the GUI opens, the database connection will immediately seed the fallback JSON files into Redis. Any PCAP you load or live traffic you capture will now successfully match against the recognized JA3 and JA4 hashes. If the GUI rendering looks slightly different than native Windows, that is just the WSL graphical architecture doing its job.

Did the interface launch properly for you, or are you hitting any X11/display server errors in the terminal?

User prompt: (.venv) diver202@DESKTOP-VF3SVQM:/mnt/e/CS331-T014-TLS_Fingerprinting\$ sudo apt install pip Reading package lists... Done Building dependency tree... Done Reading state information... Done Note, selecting 'python3-pip' instead of 'pip' The following package was automatically installed and is no longer required: libglapi-mesa Use 'sudo apt autoremove' to remove it. The following additional packages will be installed: javascript-common libexpat1-dev libjs-jquery libjs-sphinxdoc libjs-underscore libpython3-dev libpython3.12-dev python3-dev python3-wheel python3.12-dev zlib1g-dev Suggested packages: apache2 | lighttpd | httpd The following NEW packages will be installed: javascript-common libexpat1-dev libjs-jquery libjs-sphinxdoc libjs-underscore libpython3-dev libpython3.12-dev python3-dev python3-pip python3-wheel python3.12-dev zlib1g-dev 0 upgraded, 12 newly installed, 0 to remove and 81 not upgraded. Need to get 9227 kB of archives. After this operation, 41.2 MB of additional disk space will be used. Do you want to continue? [Y/n] Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 javascript-common all 11+nmu1 [5936 B] Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libexpat1-dev amd64 2.6.1-2ubuntu0.4 [140 kB] Get:3 http://archive.ubuntu.com/ubuntu noble/main amd64 libjs-jquery all 3.6.1+dfsg+~3.5.14-1 [328 kB] Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 libjs-underscore all 1.13.4~dfsg+~1.11.4-3 [118 kB] Get:5 http://archive.ubuntu.com/ubuntu noble/main amd64 libjs-sphinxdoc all 7.2.6-6 [149 kB] Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 zlib1g-dev amd64 1:1.3.dfsg-3.1ubuntu2.2 [894 kB] Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpython3.12-dev amd64 3.12.3-1ubuntu0.16 [5683 kB] Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpython3-dev amd64 3.12.3-0ubuntu2.1 [10.3 kB] Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 python3.12-dev amd64 3.12.3-1ubuntu0.16 [498 kB] Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-dev amd64 3.12.3-0ubuntu2.1 [26.7 kB] Get:11 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-wheel all 0.42.0-2 [53.1 kB] Get:12 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3-pip all 24.0+dfsg-1ubuntu1.3 [1320 kB] Fetched 9227 kB in 2s (4327 kB/s) Selecting previously unselected package javascript-common. (Reading database ... 51072 files and directories currently installed.) Preparing to unpack .../00-javascript-common_11+nmu1_all.deb ... Unpacking javascript-common (11+nmu1) ... Selecting previously unselected package libexpat1-dev:amd64. Preparing to unpack .../01-libexpat1-dev_2.6.1-2ubuntu0.4_amd64.deb ... Unpacking libexpat1-dev:amd64 (2.6.1-2ubuntu0.4) ...

```

Selecting previously unselected package libjs-jquery. Preparing to unpack
.../02-libjs-jquery_3.6.1+dfsg+~3.5.14-1_all.deb ... Unpacking libjs-jquery (3.6.1+dfsg+~3.5.14-1) ...
Selecting previously unselected package libjs-underscore. Preparing to unpack
.../03-libjs-underscore_1.13.4~dfsg+~1.11.4-3_all.deb ... Unpacking libjs-underscore
(1.13.4~dfsg+~1.11.4-3) ... Selecting previously unselected package libjs-sphinxdoc. Preparing to unpack
.../04-libjs-sphinxdoc_7.2.6-6_all.deb ... Unpacking libjs-sphinxdoc (7.2.6-6) ... Selecting previously
unselected package zlib1g-dev:amd64. Preparing to unpack
.../05-zlib1g-dev_1%3a1.3.dfsg-3.1ubuntu2.2_amd64.deb ... Unpacking zlib1g-dev:amd64
(1:1.3.dfsg-3.1ubuntu2.2) ... Selecting previously unselected package libpython3.12-dev:amd64. Preparing
to unpack .../06-libpython3.12-dev_3.12.3-1ubuntu0.16_amd64.deb ... Unpacking libpython3.12-dev:amd64
(3.12.3-1ubuntu0.16) ... Selecting previously unselected package libpython3-dev:amd64. Preparing to
unpack .../07-libpython3-dev_3.12.3-0ubuntu2.1_amd64.deb ... Unpacking libpython3-dev:amd64
(3.12.3-0ubuntu2.1) ... Selecting previously unselected package python3.12-dev. Preparing to unpack
.../08-python3.12-dev_3.12.3-1ubuntu0.16_amd64.deb ... Unpacking python3.12-dev (3.12.3-1ubuntu0.16)
... Selecting previously unselected package python3-dev. Preparing to unpack
.../09-python3-dev_3.12.3-0ubuntu2.1_amd64.deb ... Unpacking python3-dev (3.12.3-0ubuntu2.1) ...
Selecting previously unselected package python3-wheel. Preparing to unpack
.../10-python3-wheel_0.42.0-2_all.deb ... Unpacking python3-wheel (0.42.0-2) ... Selecting previously
unselected package python3-pip. Preparing to unpack .../11-python3-pip_24.0+dfsg-1ubuntu1.3_all.deb ...
Unpacking python3-pip (24.0+dfsg-1ubuntu1.3) ... Setting up javascript-common (11+nmu1) ... Setting up
python3-wheel (0.42.0-2) ... Setting up libexpat1-dev:amd64 (2.6.1-2ubuntu0.4) ... Setting up python3-pip
(24.0+dfsg-1ubuntu1.3) ... Setting up zlib1g-dev:amd64 (1:1.3.dfsg-3.1ubuntu2.2) ... Setting up libjs-jquery
(3.6.1+dfsg+~3.5.14-1) ... Setting up libjs-underscore (1.13.4~dfsg+~1.11.4-3) ... Setting up
libpython3.12-dev:amd64 (3.12.3-1ubuntu0.16) ... Setting up python3.12-dev (3.12.3-1ubuntu0.16) ...
Setting up libjs-sphinxdoc (7.2.6-6) ... Setting up libpython3-dev:amd64 (3.12.3-0ubuntu2.1) ... Setting up
python3-dev (3.12.3-0ubuntu2.1) ... Processing triggers for man-db (2.12.0-4build2) ... (.venv)
diver202@DESKTOP-VF3SVQM:/mnt/e/CS331-T014-TLS_Fingerprinting$ python -m pip install -r
code/python/requirements.txt Traceback (most recent call last): File "<frozen runpy>", line 198, in
_run_module_as_main File "<frozen runpy>", line 88, in _run_code File
"/mnt/e/CS331-T014-TLS_Fingerprinting/.venv/lib/python3.12/site-packages/pip/__main__.py", line 22, in
<module> from pip._internal.cli.main import main as _main File
"/mnt/e/CS331-T014-TLS_Fingerprinting/.venv/lib/python3.12/site-packages/pip/_internal/cli/main.py", line
10, in <module> from pip._internal.cli.autocompletion import autocomplete File
"/mnt/e/CS331-T014-TLS_Fingerprinting/.venv/lib/python3.12/site-packages/pip/_internal/cli/autocompletion.
py", line 10, in <module> from pip._internal.cli.main_parser import create_main_parser File
"/mnt/e/CS331-T014-TLS_Fingerprinting/.venv/lib/python3.12/site-packages/pip/_internal/cli/main_parser.py
", line 9, in <module> from pip._internal.build_env import get_runnable_pip File
"/mnt/e/CS331-T014-TLS_Fingerprinting/.venv/lib/python3.12/site-packages/pip/_internal/build_env.py", line
15, in <module> from pip._vendor.packaging.requirements import Requirement File
"/mnt/e/CS331-T014-TLS_Fingerprinting/.venv/lib/python3.12/site-packages/pip/_vendor/packaging/require
ments.py", line 10, in <module> from pip._vendor.pyparsing import ( # noqa File
"/mnt/e/CS331-T014-TLS_Fingerprinting/.venv/lib/python3.12/site-packages/pip/_vendor/pyparsing/__init__
.py", line 129, in <module> from .util import * ModuleNotFoundError: No module named
'pip._vendor.pyparsing.util'

```

Response: The ModuleNotFoundError occurs because your virtual environment was created before you installed the necessary system-level Python dependencies, resulting in a broken or incomplete version of pip inside the folder.

To fix this, you need to delete the corrupted environment and generate a fresh one now that Ubuntu has the

correct packages.

1. Deactivate the current environment Break out of the broken virtual environment so you can safely delete it.

deactivate

2. Delete the corrupted folder Remove the existing .venv directory completely.

```
rm -rf .venv
```

3. Recreate the virtual environment Now that python3-pip and python3.12-dev are properly installed on your WSL system, the environment module will build a complete, working version of pip.

```
python3 -m venv .venv
```

4. Reactivate the fresh environment

```
source .venv/bin/activate
```

5. Install your requirements Run your installation command again. It should now successfully download and build the dependencies.

```
python -m pip install -r code/python/requirements.txt
```

(Note: Remember to also run `python -m pip install matplotlib` if you haven't added it to your requirements text file yet, as it is needed for the new analytics panel).

User prompt: diver202@DESKTOP-VF3SVQM:/mnt/e/CS331-T014-TLS_Fingerprinting\$ rm -rf .venv
rm: cannot remove '.venv/Lib/site-packages/kiwisolver/_cext.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/matplotlib/backends/_backend_agg.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/matplotlib/ft2font.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/matplotlib/mpl-data/fonts/ttf/DejaVuSans.ttf': Permission denied
rm: cannot remove '.venv/Lib/site-packages/matplotlib/mpl-data/fonts/ttf/LastResortHE-Regular.ttf': Permission denied
rm: cannot remove '.venv/Lib/site-packages/matplotlib/_c_internal_utils.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/matplotlib/_image.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/matplotlib/_path.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/numpy/linalg/_umath_linalg.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/numpy/_core/_multiarray_umath.cp312-win_amd64.pyd': Input/output error
rm: cannot remove '.venv/Lib/site-packages/numpy.libs/libscipy_openblas64_-ed4f167a5330424524f45258e7ca2c8d.dll':

Input/output error rm: cannot remove

'./venv/Lib/site-packages/numpy.libs/msvc90/lib/libc++_1.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PIL/_imaging.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/msvc90/lib/libc++_1.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/msvc90/lib/libc++_2.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Core.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Gui.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Svg.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Widgets.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/plugins/platforms/qwindows.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/plugins/styles/qmodernwindowsstyle.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtCore.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtGui.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtSvg.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtWidgets.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/sip.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Scripts/python.exe': Input/output error
diver202@DESKTOP-VF3SVQM:/mnt/e/CS331-T014-TLS_Fingerprinting\$ cd code
diver202@DESKTOP-VF3SVQM:/mnt/e/CS331-T014-TLS_Fingerprinting/code\$ cd ..
diver202@DESKTOP-VF3SVQM:/mnt/e/CS331-T014-TLS_Fingerprinting\$ sudo rm -rf .venv rm: cannot remove
'./venv/Lib/site-packages/kiwisolver/_cext.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/matplotlib/backends/_backend_agg.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/matplotlib/ft2font.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/matplotlib/mpl-data/fonts/ttf/DejaVuSans.ttf': Permission denied rm: cannot remove
'./venv/Lib/site-packages/matplotlib/mpl-data/fonts/ttf/LastResortHE-Regular.ttf': Permission denied rm: cannot remove
'./venv/Lib/site-packages/matplotlib/_c_internal_utils.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/matplotlib/_image.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/matplotlib/_path.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/numpy/linalg/_umath_linalg.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/numpy/_core/_multiarray_umath.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/numpy.libs/libscipy_openblas64_-ed4f167a5330424524f45258e7ca2c8d.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/numpy.libs/msvc90/lib/libc++_1.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PIL/_imaging.cp312-win_amd64.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/msvc90/lib/libc++_1.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/msvc90/lib/libc++_2.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Core.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Gui.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Svg.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/bin/Qt6Widgets.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/plugins/platforms/qwindows.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/Qt6/plugins/styles/qmodernwindowsstyle.dll': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtCore.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtGui.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtSvg.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/QtWidgets.pyd': Input/output error rm: cannot remove
'./venv/Lib/site-packages/PyQt6/sip.cp312-win_amd64.pyd': Input/output error rm: cannot remove

'.venv/Scripts/python.exe': Input/output error

Response: The Input/output error and Permission denied messages occur because Windows is currently holding file locks on those specific binaries. Because your project is located on a mounted Windows drive (/mnt/e/) and the existing .venv was originally created using Windows Python (indicated by the win_amd64.pyd and .exe extensions), WSL cannot delete them while a host Windows process is actively using them.

Steps to unlock and delete the folder

- Close the Python GUI if it is still running on your Windows desktop.
- Close any Windows Command Prompt, PowerShell, or IDEs (like VS Code) that might have the .venv activated or the project folder open.
- Open your Windows Task Manager, locate any lingering python.exe processes, and end them.
- Return to your WSL terminal and execute `rm -rf .venv` again.

Once the Windows locks are cleared, the folder will delete successfully. You can then create the fresh, Linux-native virtual environment using `python3 -m venv .venv`, activate it with `source .venv/bin/activate`, and run your pip installation command.

User prompt: Interface launched, it's in light mode somehow, and these are the only two interfaces it shows :(

Response: The reason you are seeing a light theme and only the lo and eth0 interfaces is because WSL is a fully isolated virtual machine running its own display server and virtualized network.

Why this happens

- **The Interfaces:** WSL2 operates behind a virtual NAT switch. The lo interface is the local loopback, and eth0 is the virtual Linux adapter. WSL inherently cannot access or set promiscuous mode on your physical Windows Wi-Fi or Ethernet adapters to capture live host traffic.
- **The Theme:** The WSL graphics layer (WSLg) utilizes default Linux GTK/Qt UI rendering. It does not pull your Windows registry settings to detect that your host system is running dark mode.

To capture live traffic from your actual Windows system while keeping the fingerprint database working, you can use a hybrid approach. Because WSL automatically bridges localhost ports to Windows, you can leave the Redis database running quietly in the background on Ubuntu, but run your Python GUI natively on Windows. This grants you native dark mode and direct Scapy access to your physical network cards.

Step 1: Install Windows Packet Capture Download and install Npcap on your Windows machine. During the installation wizard, you must check the box for "Install Npcap in WinPcap API-compatible Mode" so Scapy can properly hook into your interfaces.

Step 2: Setup the Native Windows Environment Open a standard Windows Command Prompt or PowerShell (not your Ubuntu terminal) and navigate to your CS331-T014-TLS_Fingerprinting folder on your E: drive. Since the current .venv folder contains Linux binaries, leave it alone and create a dedicated Windows environment:

- `python -m venv .venv_win`

- `.venv_win\Scripts\activate`
- `pip install -r code\python\requirements.txt`
- `pip install matplotlib`

Step 3: Run the GUI With your Windows virtual environment active, launch the application:

- `python code\gui\main.py`

Because your WSL instance is still actively running Redis on 127.0.0.1:6379, the Windows Python application will seamlessly connect to it through the local port forward. The PyQt6 window will now natively match your dark system theme, and the interface dropdown will populate with all your physical Windows network adapters.

User prompt: Here is the GUI folder. Drastic changes need to happen. The analytics thing breaks on different screen sizes. It pushes the buttons out so you cannot use them. Restructure everything. All I want is on clicking Analytics, I want a separate window to open, which can be closed like any other window. This will only have the pie chart. Remove the line chart completely. Also I want the pie chart to look nice as fuck. Please do this correctly/

Response: The layout has been flattened to remove the sidebar behavior entirely, and the analytics view is now spawned as a detached, top-level window. The line chart and animation logic have been stripped out, while the pie chart has been rebuilt into a modern, system-themed donut chart with custom color palettes and a clean legend.

analytics.py

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout
from PyQt6.QtGui import QPalette, QIcon
from PyQt6.QtCore import Qt
from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg
from matplotlib.figure import Figure
from collections import Counter
```

```
class AnalyticsWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Analytics - Client Distribution")
        self.resize(700, 500)
        self.setWindowFlag(Qt.WindowType.Window)

        self.layout = QVBoxLayout(self)
        self.layout.setContentsMargins(10, 10, 10, 10)

        self.figure = Figure(figsize=(6, 5))
        self.canvas = FigureCanvasQTAgg(self.figure)
        self.layout.addWidget(self.canvas)

        self.pieAx = self.figure.add_subplot(111)
        self.rowCacheRef = []
```

```

# A sleek, modern color palette for the donut chart
self.chartColors = [
    "#47B39C", "#FFC154", "#EC6B56", "#457B9D",
    "#E63946", "#008080", "#1D3557", "#A8DADC",
    "#F4A261", "#2A9D8F", "#E9C46A", "#264653"
]

def setRowCache(self, rowCache):
    self.rowCacheRef = rowCache

def refreshCharts(self):
    self.pieAx.clear()

    bgColor = self.palette().color(QPalette.ColorRole.Window).name()
    textColor = self.palette().color(QPalette.ColorRole.WindowText).name()

    self.figure.patch.set_facecolor(bgColor)
    self.pieAx.set_facecolor(bgColor)

    if not self.rowCacheRef:
        self.canvas.draw()
        return

    matches = []
    for row in self.rowCacheRef:
        match = row.get("ja3Match", "Unknown")
        if match == "Unknown":
            match = row.get("ja4Match", "Unknown")
        matches.append(match)

    pieCounts = Counter(matches)
    labels = list(pieCounts.keys())
    sizes = list(pieCounts.values())

    if sizes:
        wedges, texts, autotexts = self.pieAx.pie(
            sizes,
            autopct='%1.1f%%',
            startangle=90,
            pctdistance=0.75,
            colors=self.chartColors,
            wedgeprops=dict(width=0.4, edgecolor=bgColor, linewidth=2),
            textprops={'color': textColor, 'weight': 'bold', 'fontsize': 10}
        )

        self.pieAx.set_title("Client Fingerprint Distribution", color=textColor, weight='bold', pad=20,
            fontsize=14)

        legend = self.pieAx.legend(
            wedges, labels,
            title="Identified Clients",

```

```

        loc="center left",
        bbox_to_anchor=(0.9, 0, 0.5, 1),
        frameon=False,
        labelcolor=textColor
    )
    legend.get_title().set_color(textColor)
    legend.get_title().set_weight("bold")

self.figure.tight_layout()
self.canvas.draw()

```

window.py

```

from pathlib import Path
import shutil
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QColor
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
    QTableWidgetItem, QTableWidgetItem, QFileDialog, QHeaderView,
    QComboBox, QLabel, QTextEdit, QInputDialog, QAbstractItemView,
    QLineEdit
)
from scapy.all import get_working_ifaces, wrpcap
from src.db import FingerprintDB
from workers import PcapWorker, LiveCaptureWorker
from analytics import AnalyticsWindow

class TlsMonitorGui(QMainWindow):
    def __init__(self, pythonEngineDir):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1150, 650)
        self.dbPath = pythonEngineDir / "fingerprints.json"
        self.pcapsDir = pythonEngineDir.parent / "pcaps"
        self.fingerprintDb = None
        self.pcapWorker = None
        self.liveWorker = None
        self.rowCache = []
        self.capturedPackets = []
        self.loadedPcapPath = None

        self.analyticsWindow = AnalyticsWindow()
        self.analyticsWindow.setRowCache(self.rowCache)

        mainWidget = QWidget()
        self.setCentralWidget(mainWidget)
        mainLayout = QVBoxLayout(mainWidget)

        controlLayout = QHBoxLayout()

```

```

self.loadButton = QPushButton("Load PCAP")
self.loadButton.clicked.connect(self.selectPcapFile)
controlLayout.addWidget(self.loadButton)

self.cancelLoadButton = QPushButton("Cancel Load")
self.cancelLoadButton.clicked.connect(self.cancelPcapLoad)
self.cancelLoadButton.setEnabled(False)
controlLayout.addWidget(self.cancelLoadButton)

self.saveButton = QPushButton("Save PCAP")
self.saveButton.clicked.connect(self.savePcapFile)
controlLayout.addWidget(self.saveButton)

self.clearButton = QPushButton("Clear Records")
self.clearButton.clicked.connect(self.clearRecords)
controlLayout.addWidget(self.clearButton)

self.refreshInterfacesButton = QPushButton("Refresh Interfaces")
self.refreshInterfacesButton.clicked.connect(self.populateInterfaces)
controlLayout.addWidget(self.refreshInterfacesButton)

controlLayout.addWidget(QLabel("Interface:"))
self.interfaceDropdown = QComboBox()
self.populateInterfaces()
controlLayout.addWidget(self.interfaceDropdown)

controlLayout.addWidget(QLabel("BPF Filter:"))
self.bpfFilterInput = QLineEdit("tcp port 443")
controlLayout.addWidget(self.bpfFilterInput)

self.liveButton = QPushButton("Start Live Capture")
self.liveButton.clicked.connect(self.toggleLiveCapture)
controlLayout.addWidget(self.liveButton)

self.labelButton = QPushButton("Label Fingerprint")
self.labelButton.clicked.connect(self.labelSelectedFingerprint)
self.labelButton.setEnabled(False)
controlLayout.addWidget(self.labelButton)

self.analyticsButton = QPushButton("Analytics")
self.analyticsButton.clicked.connect(self.openAnalytics)
controlLayout.addWidget(self.analyticsButton)

controlLayout.addStretch()
mainLayout.addLayout(controlLayout)

self.dataTable = QTableWidgetItem(0, 8)
self.dataTable.setHorizontalHeaderLabels([
    "Role", "Source", "Destination", "SNI", "JA3 Hash", "JA4 Hash",
    "JA3 Match", "JA4 Match"
])
])

```

```

header = self.dataTable.horizontalHeader()
header.setSectionResizeMode(QHeaderView.ResizeMode.ResizeToContents)
header.setSectionResizeMode(3, QHeaderView.ResizeMode.Stretch)
self.dataTable.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows)
self.dataTable.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
self.dataTable.itemSelectionChanged.connect(self.displayRowDetails)
mainLayout.addWidget(self.dataTable)

detailsLayout = QHBoxLayout()
self.detailsPane = QTextEdit()
self.detailsPane.setReadOnly(True)
self.detailsPane.setFixedHeight(120)
self.detailsPane.setPlaceholderText("Select any handshake above to view raw metadata and JA3
string.")
detailsLayout.addWidget(self.detailsPane)

self.timestampLabel = QLabel("Timestamp: N/A")
self.timestampLabel.setAlignment(Qt.AlignmentFlag.AlignBottom | Qt.AlignmentFlag.AlignRight)
detailsLayout.addWidget(self.timestampLabel)

mainLayout.addLayout(detailsLayout)

def openAnalytics(self):
    self.analyticsWindow.refreshCharts()
    self.analyticsWindow.show()
    self.analyticsWindow.raise_()
    self.analyticsWindow.activateWindow()

def updateAnalyticsIfVisible(self):
    if self.analyticsWindow.isVisible():
        self.analyticsWindow.refreshCharts()

def populateInterfaces(self):
    self.interfaceDropdown.clear()
    self.interfaceDropdown.addItem("Any (All Interfaces)", "Any")
    try:
        for iface in get_working_ifaces():
            self.interfaceDropdown.addItem(iface.name, iface.network_name)
    except Exception:
        self.interfaceDropdown.addItem("Default", "Default")

def closeEvent(self, event):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.pcapWorker.wait()
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveWorker.wait()
    self.analyticsWindow.close()
    event.accept()

def getDb(self):

```

```

if self.fingerprintDb is None:
    self.fingerprintDb = FingerprintDB(str(self.dbPath))
return self.fingerprintDb

def selectPcapFile(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        return
    filePath, _ = QFileDialog.getOpenFileName(
        self, "Open PCAP", str(self.pcapsDir), "PCAP Files (*.pcap *.pcapng)"
    )
    if filePath:
        self.clearRecords()
        self.loadedPcapPath = filePath
        self.loadButton.setEnabled(False)
        self.liveButton.setEnabled(False)
        self.cancelLoadButton.setEnabled(True)
        self.pcapWorker = PcapWorker(filePath, self.dbPath)
        self.pcapWorker.rowExtracted.connect(self.appendTableRow)
        self.pcapWorker.errorOccurred.connect(lambda err: self.detailsPane.setText(f"Error: {err}"))
        self.pcapWorker.captureFinished.connect(self.onCaptureFinished)
        self.pcapWorker.start()

def cancelPcapLoad(self):
    if self.pcapWorker and self.pcapWorker.isRunning():
        self.pcapWorker.stopCapture()
        self.detailsPane.append("\nPCAP loading cancelled.")
        self.updateAnalyticsIfVisible()

def onCaptureFinished(self, statsSummary):
    self.loadButton.setEnabled(True)
    self.liveButton.setEnabled(True)
    self.cancelLoadButton.setEnabled(False)
    currentText = self.detailsPane.toPlainText()
    self.detailsPane.setText(f"{currentText}\n\nCapture Finished.\n{statsSummary}")
    self.updateAnalyticsIfVisible()

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning():
        self.liveWorker.stopCapture()
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        if self.pcapWorker and self.pcapWorker.isRunning():
            return
        selectedInterface = self.interfaceDropdown.currentData()
        currentFilter = self.bpfFilterInput.text()
        self.clearRecords()
        self.liveWorker = LiveCaptureWorker(
            selectedInterface, self.dbPath, currentFilter, packetBuffer=self.capturedPackets
        )
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.errorOccurred.connect(

```

```

        lambda errorMsg: self.detailsPane.setText(errorMsg)
    )
    self.liveWorker.captureFinished.connect(self.onCaptureFinished)
    self.liveWorker.start()
    self.liveButton.setText("Stop Live Capture")
    self.loadButton.setEnabled(False)
    self.cancelLoadButton.setEnabled(False)

def clearRecords(self):
    self.dataTable.setRowCount(0)
    self.rowCache.clear()
    self.capturedPackets.clear()
    self.loadedPcapPath = None
    self.detailsPane.clear()
    self.timestampLabel.setText("Timestamp: N/A")
    self.labelButton.setEnabled(False)
    self.updateAnalyticsIfVisible()

def savePcapFile(self):
    if not self.capturedPackets and not self.loadedPcapPath:
        self.detailsPane.setText("No captured or loaded packets available to save.")
        return
    savePath, _ = QFileDialog.getSaveFileName(
        self, "Save PCAP", str(self.pcapsDir / "captured.pcap"), "PCAP Files (*.pcap *.pcapng)"
    )
    if not savePath:
        return
    try:
        if self.capturedPackets:
            packetsSnapshot = list(self.capturedPackets)
            wrpcap(savePath, packetsSnapshot)
            self.detailsPane.setText(f"Successfully saved {len(packetsSnapshot)} packet(s) to {savePath}")
        elif self.loadedPcapPath:
            shutil.copyfile(self.loadedPcapPath, savePath)
            self.detailsPane.setText(f"Successfully exported PCAP to {savePath}")
    except Exception as err:
        self.detailsPane.setText(f"Failed to save PCAP: {err}")

def appendTableRow(self, packetData):
    if "captureTime" not in packetData:
        packetData["captureTime"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]

    currentRow = self.dataTable.rowCount()
    self.dataTable.insertRow(currentRow)
    self.rowCache.append(packetData)
    columnValues = [
        packetData["role"],
        packetData["source"],
        packetData["destination"],
        packetData["sni"],
        packetData["ja3Hash"],
        packetData["ja4Hash"],

```

```

        packetData["ja3Match"],
        packetData["ja4Match"],
    ]
    isClient = packetData["role"] == "Client"
    rowColor = QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20)
    for colIdx, val in enumerate(columnValues):
        item = QTableWidgetItem(str(val))
        item.setFlags(item.flags() & ~Qt.ItemFlag.ItemIsEditable)
        item.setBackground(rowColor)
        self.dataTable.setItem(currentRow, colIdx, item)
    self.dataTable.scrollToBottom()
    self.updateAnalyticsIfVisible()

def displayRowDetails(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        self.labelButton.setEnabled(False)
        return
    rowIndex = selectedRows[0].row()
    if rowIndex < len(self.rowCache):
        entry = self.rowCache[rowIndex]
        isUnknown = entry.get("ja3Match") == "Unknown" or entry.get("ja4Match") == "Unknown"
        self.labelButton.setEnabled(not isUnknown)
        detailText = (
            f"Role:      {entry['role']}\n"
            f"SNI:      {entry['sni']}\n"
            f"JA3 Hash:  {entry['ja3Hash']}\n"
            f"JA3 Match: {entry['ja3Match']}\n"
            f"JA4 Hash:  {entry['ja4Hash']}\n"
            f"JA4 Match: {entry['ja4Match']}\n"
            f"Raw JA3:   {entry.get('ja3Raw', 'N/A')}"
        )
        self.detailsPane.setText(detailText)

        capTime = entry.get("captureTime", "N/A")
        self.timestampLabel.setText(f"Timestamp:\n{capTime}")

def labelSelectedFingerprint(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        return
    rowIndex = selectedRows[0].row()
    if rowIndex >= len(self.rowCache):
        return
    entry = self.rowCache[rowIndex]
    if entry.get("ja3Match") != "Unknown" and entry.get("ja4Match") != "Unknown":
        return
    label, accepted = QInputDialog.getText(
        self,
        "Label Fingerprint",
        "Verified name:",
    )

```

```

if not accepted or not label.strip():
    return
try:
    db = self.getDb()
    pair = (
        ("ja3", "ja3Hash", "ja3Match", 6),
        ("ja4", "ja4Hash", "ja4Match", 7),
    )
    if entry.get("fingerprintKind") in ("ja3s", "ja4s"):
        pair = (
            ("ja3s", "ja3Hash", "ja3Match", 6),
            ("ja4s", "ja4Hash", "ja4Match", 7),
        )
    for kind, hashKey, matchKey, _ in pair:
        if entry.get(matchKey) == "Unknown":
            db.enroll(entry[hashKey], kind, label)
except (OSError, ValueError) as error:
    self.detailsPane.setText(f"Could not save fingerprint: {error}")
    return
for matchKey, targetColumn in (("ja3Match", 6), ("ja4Match", 7)):
    if entry.get(matchKey) == "Unknown":
        entry[matchKey] = label.strip()
        newItem = QTableWidgetItem(entry[matchKey])
        newItem.setFlags(newItem.flags() & ~Qt.ItemFlag.ItemIsEditable)
        isClient = entry["role"] == "Client"
        newItem.setBackground(QColor(0, 255, 0, 20) if isClient else QColor(0, 0, 255, 20))
        self.dataTable.setItem(rowIndex, targetColumn, newItem)
self.displayRowDetails()
self.updateAnalyticsIfVisible()

```
